MATH IN SOCIETY / MODULE 9

Computing with Two Choices

Binary numbers, decision trees, and short programs

A question for the week

A community center needs to direct callers to the right service. It could ask everyone the same two questions, or it could use a branching menu that takes fewer questions for common requests. How can a few yes-or-no choices describe numbers, routes, and useful instructions?

We will start by counting with two symbols. Then we will follow branches in a tree, build a small decision program, count the work it requires, and ask what happens when we keep extending the tree. The calculations use counting, addition, and multiplication. You do not need to know how to code.

1. Counting with zeros and ones

Our usual decimal system has ten digits. Its positions count ones, tens, hundreds, and so on. The **binary** system has two digits: 0 and 1. Each position is worth twice the one to its right.

Position	5th	4th	3rd	2nd	1st
Worth	16	8	4	2	1

For example, the binary word `1101` represents $8+4+0+1=13$ in decimal. The leftmost 1 uses the 8's place, the next 1 uses the 4's place, the 0 skips the 2's place, and the last 1 uses the 1's place.

From decimal 11 to binary

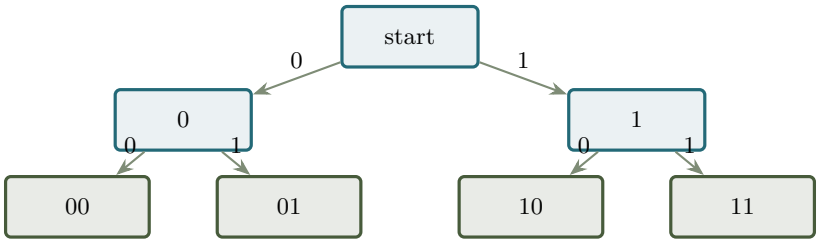
Find the largest power of two not exceeding 11: that is 8. We have 3 left, so use 2 and 1 as well. The place values 8, 4, 2, 1 receive digits $1, 0, 1, 1$. Thus $11 = 1011_2$.

Try it

(a) Find the decimal value of 10110_2 using the place values $16, 8, 4, 2, 1$. (b) Write 14 in binary. (c) How many distinct three-digit strings of zeros and ones are possible if leading zeros are allowed? (Here a *string* is a code, not necessarily the usual way to write a number.)

2. A code can be a route

Suppose 0 means "go left" and 1 means "go right." Beginning at a dot, each choice follows one branch. A string like `101` tells you exactly which route to take: right, left, right.



The four addresses after two choices are `00`, `01`, `10`, and `11`. Every additional choice doubles the number of possibilities. With one choice there are 2 addresses; with two there are 4; with three there are 8; with four there are 16. In general, there are 2^n binary strings with exactly n digits.

Find the address

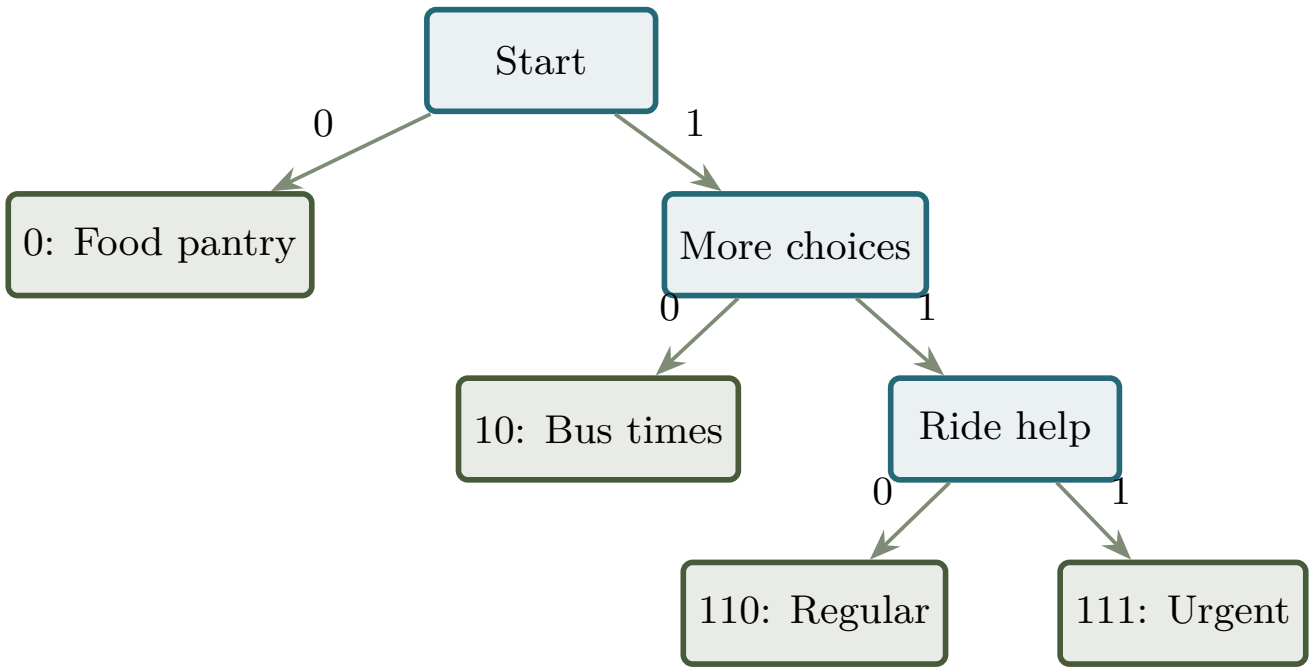
The address `011` means left, then right, then right. The first two digits `01` bring you to a dot in the two-level tree. A third choice continues from that dot. That first part, `01`, is called a **prefix**.

Try it

(a) List all eight three-digit binary addresses. (b) How many four-digit addresses begin with `10`? (c) Is `10` a prefix of `1011`? Is `11`?

3. Turning a tree into a program

A community center answers questions about food pantry hours, bus schedules, regular ride help, and urgent ride help. A caller enters 0 or 1 at each junction and stops when a service appears.



This is a **finite decision program**: follow the branch named by each digit until you arrive at an instruction to stop and show a service. It has four stopping codes:

Service	Code	Questions answered
Food pantry	`0`	1
Bus times	`10`	2
Regular ride help	`110`	3
Urgent ride help	`111`	3

Notice that `0` is never the beginning of another *complete* code. Neither is `10`. This is a **prefix-free code**: no stopping code is an initial part of another stopping code. As soon as a caller reaches a service, the system knows the code is complete.

Try it

(a) Trace the code `110`. Which service answers? (b) Why would using `1` as an additional stopping code cause a problem? (c) Design a new two-question menu for four services, with a distinct two-digit code for each service.

4. Counting the work a program does

A decision menu is useful when it answers common requests quickly. Suppose the center receives 16 calls:

Service	Calls	Questions per call	Total questions
Food pantry	8	1	8
Bus times	4	2	8
Regular ride help	2	3	6
Urgent ride help	2	3	6
All calls	16		28

The total is $8(1) + 4(2) + 2(3) + 2(3) = 28$ questions. If every caller answered exactly two questions, the total would be $16(2) = 32$. For this group of callers, the branching menu saves four questions.

Same tree, different callers

A different community center might receive urgent requests most often. The tree's shape can stay the same while the service labels change. Assigning the shortest code to the most frequent request changes how much work the menu does.

Try it

Suppose the call counts are 6 for food pantry, 5 for bus times, 3 for regular ride help, and 2 for urgent help. How many questions does the current menu ask in total? How many would a two-question menu ask? Which one uses fewer questions?

5. A tree that keeps growing

A decision program can stop after a few choices. But we can also imagine a binary tree with no final level: every address has two longer addresses beneath it, one ending in 0 and one ending in 1. A three-digit address is a location *within* that tree, not the end of the whole tree.

Choices made	0	1	2	3	4
Possible addresses	1	2	4	8	16

An **infinite path** would be a never-ending sequence of choices. At any finite stage we know only its beginning, called a **finite prefix**. For example, after seeing `101`, the next digit could be 0 or 1. Both paths agree on everything observed so far.

A rule removes some possibilities

Imagine a code that never uses two 1s together. The allowed two-digit strings are `00`, `01`, and `10`. The string `11` is forbidden. At three digits, the allowed strings are `000`, `001`, `010`, `100`, `101`: five addresses. At four digits there are eight.

Try it

(a) List the allowable three-digit strings that begin with 1. (b) Write two allowed four-digit strings and one forbidden four-digit string. (c) Without listing everything, explain why every allowed string ending in 1 must be followed by 0 if we extend it one step.

The phone menu occupies a **finite subtree** of the full binary tree: it keeps the junctions needed to reach the four services, with a stopping instruction at each service. This decision program always finishes after a limited number of questions. More general programs can repeat instructions in loops, so a short program may run for many steps. That distinction is one reason the study of computation is interesting.

6. What have we discovered?

Binary digits can stand for numbers or for instructions. The meaning comes from the rule we choose. In a decision tree, a code is a path. A stopping instruction makes the tree into a simple program. By counting the paths and questions, we can compare how much work different designs require. Even a short rule can organize an ever-growing set of possible routes.

One reliable way to investigate a decision program

(1) Say what each choice means. (2) Trace the path for a particular input. (3) Identify when the process stops. (4) Count the questions or possible outcomes. (5) Explain one detail that the model leaves out.

Practice

1. Convert 11010_2 to decimal. Write 9 in binary.
2. How many addresses have exactly five binary digits? How many of them begin with `011`?
3. In the menu above, which code reaches urgent ride help? Why can the code `1` not also mean "stop"?
4. A desk receives 10 pantry requests, 3 bus requests, 2 regular ride requests, and 1 urgent request. Count the total questions using the menu above. Compare with a fixed two-question menu.
5. List all allowed two-digit and three-digit codes that avoid `11`. Explain why the number is smaller than 2^n.
6. Describe a useful decision tree from daily life. Name one thing the tree would need to know that is not represented by a simple yes-or-no choice.

Answers to calculation questions

1. $11010_2 = 16+8+2 = 26$ and $9 = 1001_2$. 2. $2^5 = 32$ total; four begin with `011` because two positions remain free. 3. `111`; stopping at `1` would prevent distinguishing the three codes that begin with 1. 4. $10(1)+3(2)+2(3)+1(3) = 25$ questions, versus $16(2) = 32$. 5. Two digits: `00`, `01`, `10`; three: `000`, `001`, `010`, `100`, `101`. The rule removes all addresses with adjacent 1s. 6. Answers vary.